Distributed Systems Simplified

Distributed Caching (LRU & TTL):

Turning milliseconds into microseconds..

AUTHOR:

Niv Rave

Github: https://github.com/NivRave/Distributed-Systems-Simplified

The Latency Problem

The Database Bottleneck

Databases are great for persistence, but they are relatively slow.

The Problem:
Repeatedly querying the disk for the same "Hot Data" (like a trending news story) wastes CPU and slows down your users.

The Result:
High latency and a database that's constantly under 100% load.

The Scaling Wall:

- **Disk I/O:** No matter how many replicas you have, hitting the disk takes milliseconds.
- **Cost:** Scaling a DB cluster to handle a traffic spike is expensive and slow.

The Fix: Caching. Store the result in fast RAM for the next request.

The Architectural Solution

The Cache-Aside Pattern

In our simulation, we implement the Cache-Aside strategy:

1. Cache Hit: The app finds the data in RAM. Response time: < 1ms.

2. Cache Miss: The app queries the DB, saves the result in the cache, and returns it. Response time: 50ms+.

Memory Management:

- **LRU (Least Recently Used):** When the cache is full, we evict the item that hasn't been touched in the longest time.
- **TTL (Time-To-Live):** We set an expiration date on data to ensure users don't see stale information.

Code: The LRU Structure

Modeling the Fast Storage

We combine a *map* for O(1) lookups with a *doubly linked list* for O(1) eviction tracking.

- ***container/list*:** This standard Go package allows us to move "recently used" items to the front of the line instantly.
- ***CacheItem*:** Each entry stores its value and its specific expiration timestamp.

Below is the implementation under *cache.go*:

```go
type LRUCache struct {
    capacity  int
    items     map[string]*list.Element
    evictList *list.List
    mu        sync.Mutex
}

type CacheItem struct {
    Value     string
    ExpiresAt time.Time
}
```

Code: The "Get" Logic

O(1) Access & TTL Check

Every time we access a key, we must check if it's still "Fresh".

- **The Action:** If the item exists but *time.Now()* is past *ExpiresAt*, we purge it immediately.
- **LRU Update:** If it's a hit, we call *MoveToFront*. This ensures active keys are never evicted.

Below is the implementation under *cache.go*:

```go
func (c *LRUCache) Get(key string) (string, bool) {
    c.mu.Lock()
    defer c.mu.Unlock()

    if el, ok := c.items[key]; ok {
        item := el.Value.(*CacheItem)

        // Is the data stale?
        if time.Now().After(item.ExpiresAt) {
            c.evictList.Remove(el)
            delete(c.items, key)
            return "", false
        }

        c.evictList.MoveToFront(el) // Still "Hot"
        return item.Value, true
    }
    return "", false
}
```

Code: Eviction Strategy

Managing Expensive RAM

When we reach capacity, we must "forget" the oldest data to make room for the new.

- **The Push:** New items go to the Front of the list.
- **The Evict:** If we exceed capacity, we remove the item at the Back of the list.

Below is the implementation under *cache.go*:

```go
func (c *LRUCache) Set(key, value string, ttl time.Duration) {
    c.mu.Lock()
    defer c.mu.Unlock()

    // 1. Add new item to the Front (MRU)
    item := &CacheItem{Value: value, ExpiresAt: time.Now().Add(ttl)}
    el := c.evictList.PushFront(item)
    c.items[key] = el

    // 2. If full, remove the oldest from the Back (LRU)
    if c.evictList.Len() > c.capacity {
        oldest := c.evictList.Back()
        c.evictList.Remove(oldest)
        delete(c.items, oldest.Value.(*CacheItem).Key)
    }
}
```

Code: Visualizing the Execution

The Simulation in Action - Try it out!

We orchestrate three scenarios to prove the speed and reliability of our cache.

- **Scenario 1:** The "Hot Key" - Watch latency drop from 50ms to 0.1ms.
- **Scenario 2:** LRU Eviction - Observe the cache dropping the oldest item when we hit capacity=3.
- **Scenario 3:** TTL Expiration - See the cache force a DB refresh after 1 second to keep data accurate.

Below is the implementation under *simulation.go*:

```go
func (s *Simulation) FetchData(key string, ttl time.Duration) string {
    start := time.Now()

    // 1. Check RAM (Fast)
    val, hit := s.Cache.Get(key)
    if hit {
        fmt.Printf("[CACHE HIT] Latency: %v\n", time.Since(start))
        return val
    }

    // 2. Fallback to Disk (Slow)
    val = s.DB.Read(key)
    s.Cache.Set(key, val, ttl)
    return val
}
```

Code: Performance Analysis

Stress Testing Data Freshness

The terminal output clearly shows the trade-offs we've made.

```
-----------------------------------------------------
Scenario 1: The Concept of a 'Hot Key'
Goal: Show dramatic latency reduction on repeated reads
-----------------------------------------------------

[Step 1] First time reading (DB Query required).
   [CACHE MISS] Key: trending_news:1 | Latency: 50.562ms (Fetched from DB)

[Step 2] Next 4 reads: Item is in cache. Loads instantly.
   [CACHE HIT]  Key: trending_news:1 | Latency: 0s
   [CACHE HIT]  Key: trending_news:1 | Latency: 0s
   [CACHE HIT]  Key: trending_news:1 | Latency: 0s
   [CACHE HIT]  Key: trending_news:1 | Latency: 0s

-----------------------------------------------------
Scenario 2: LRU Eviction (Memory Management)
Goal: Show that cache purges old items when full (Capacity = 3)
-----------------------------------------------------

[Step 1] Insert 3 items into Cache (Fills Capacity).
   [CACHE MISS] Key: item:A         | Latency: 50.1581ms (Fetched from DB)
   [CACHE MISS] Key: item:B         | Latency: 50.7512ms (Fetched from DB)
   [CACHE MISS] Key: item:C         | Latency: 50.5557ms (Fetched from DB)

[Cache State] Ordered Most-Recently-Used -> Least-Recently-Used:
   Keys: [item:C item:B item:A]

[Step 2] Fetch a new item:D. This will evict the oldest (item:A).
   [CACHE MISS] Key: item:D         | Latency: 50.4979ms (Fetched from DB)
   Keys: [item:D item:C item:B] <- Notice item:A is gone

[Step 3] Try to read item:A again.
   [CACHE MISS] Key: item:A         | Latency: 50.3668ms (Fetched from DB)

-----------------------------------------------------
Scenario 3: TTL (Time-To-Live) Expiration
Goal: Ensure data freshness by expiring old data
-----------------------------------------------------

[Step 1] Fetch price for the first time.
   [CACHE MISS] Key: stock_price:TSLA | Latency: 50.286ms (Fetched from DB)

[Step 2] Read immediately again (Data is fresh).
   [CACHE HIT]  Key: stock_price:TSLA | Latency: 0s

[Step 3] Wait 1.1 seconds for TTL to expire...

[Step 4] Read again it after TTL. Cache should purge it and fetch anew.
   [CACHE MISS] Key: stock_price:TSLA | Latency: 50.9542ms (Fetched from DB)
```

Summary & Takeaways

CACHE-ASIDE IS KING

It's the most resilient pattern—if the cache dies, the app still works (just slower).

LRU IS ESSENTIAL

RAM is a finite resource. You must have a strategy for what to delete when you're full.

TTL PREVENTS STALE DATA

Never cache forever. Stale data is often worse than slow data.

O(1) COMPLEXITY

By using a Map + Linked List, your cache speed stays the same whether you have 10 items or 10 million.

Want to dive deeper?

Explore the full source code and deep-dive documentation on my GitHub repository or connect via LinkedIn.

GitHub LinkedIn

LIKE • COMMENT • SHARE